

HW3 Strategy — GSP Ad-Auction Bidding Bots

Environment. A repeated GSP auction (no ad quality): each round we submit a bid, the engine ranks bids and assigns slots, and the winner of slot j pays the next-highest bid; our round utility is $(\text{value} - \text{price}) \cdot \text{CTR}[\text{slot}]$. The key fact we exploit: a bid sets our rank, not our price - the price is whatever the agent directly below us bid. From the public per-round results (agent_id, slot, price) we recover rivals' bids almost exactly, because the price paid for slot j equals the raw bid of the agent in slot $j+1$ - so this is system identification + best response, not a bandit. We accumulate the history ourselves (the grader hands us only the current round), and all bookkeeping is $O(\text{slots})$ per call, far inside the 50 ms budget. Every bid is a finite float in $[0, \text{budget}]$.

Agent 1 (no budget) - distribution-aware expected-utility, with a targeted descent. Rather than best-responding to only the last round, we accumulate each rival's reconstructed-bid distribution across rounds (count, mean, spread, max, last) and model each as a few plausible bids $\{\text{mean}, \text{mean} - \text{sd}, \text{mean} + \text{sd}\}$. We then choose the bid that maximizes expected utility over the joint product of those scenarios - comparing every reachable slot honestly against the free bottom slot (bid 0, pay nothing) and never targeting a negative-utility slot. Being distribution-aware is what tames the random bidder: we respond to its whole range instead of chasing whichever draw it happened to make last round. Three targeted overrides. Against a constant rival - a fixed-fraction bidder we can pin exactly, since the price of the slot above any non-top agent reveals its bid, so one observation identifies it. (1) The descent: when our expected-utility-optimal seat is the top slot sitting directly above that constant rival, we evaluate dropping to just below it and take the descent only if it widens our expected margin versus that rival. (2) The cost-raise: when our best seat is instead second, directly below the constant rival, we raise our bid to just under its identified bid. Against the stochastic rival - which bids $u \cdot v$ with $u \sim U(0.4, 1)$ and so cannot be pinned to a point - (3) a censoring-safe cost-raise: its observed bids and the prices it pays lower-bound its scale v_{lb} , so when it sits directly above us in slot 1 we raise our bid toward $0.7 \cdot v_{\text{lb}}$. In GSP our own price is set by whoever sits below us, so all three moves make the rival above pay near its full bid while our own price is untouched - its utility falls and our relative standing rises; the constant cost-raise never overtakes (seat unchanged), while the stochastic one trades a little own-utility for a stronger hit on the hardest rival. We open value-anchored before any history and fall back to a point estimate when the field is large (guarding latency and the many-agent regime). (A value-agnostic suppression objective and several weightings were measured on a Common-Random-Numbers screen and rejected as over-suppressing; the targeted, type-gated descent and cost-raise were the versions that widened our margin without eroding own utility.)

Agent 2 (budget) - best-response + self-correcting pacing. We compute the same best-response target, then modulate it by how far ahead/behind we are on budget versus time:
$$\text{bid} = \text{best_response} \times \min(1.5, (\text{budget_remaining} / \text{total_budget}) / (\text{rounds_left} / T))$$
When we are ahead on budget we press; when behind we retreat to cheaper slots - so spend spreads across all T rounds. Because leftover budget is worthless we use it (the multiplier grows late as rounds_left shrinks), but we never buy a negative-marginal-utility click merely to exhaust it, and the budget also caps every bid.

Robustness. Strategies were selected on a Common-Random-Numbers tournament over the real grader (paired, many simulations) across both the 4-agent and many-agent fields - not a noisy small-sample run.